The Reflection Tree Contender

A constructive, axiom-free Lean entry exceeding Contender 5

A reproducible Lean 4 submission

13 October 2026

Abstract

We adapt Cody Roux's constructive biggest-number contest from Coq to Lean and give a closed, executable Lean natural number N. The previous contender, denoted C_5, is the maximum produced by the repository's total interpreter over all raw terms of depth at most 42. We preserve that interpreter's behavior on open and malformed terms, then build an oracle language containing literal quotations of its syntax. A bounded semantic maximum gives a constructive diagonal operator $\mathcal{R}$; structural recursion on countably branching trees iterates this operator. An explicit tree construction combines Veblen-inspired operations, amplification at every finite functional type, and iteration whose index is itself an amplified tree.

Lean checks $C_5 < N$ without evaluating either number. More quantitatively, if $x_0 = C_5$ and $x_{j+1} = 2^{x_j+44}$, Lean checks $x_5 \le N$. We also prove that $\mathcal{R}(f)$ eventually strictly exceeds the output of every fixed program in the language with oracle f, and cannot be represented there even on a final segment of the inputs. All audited declarations have an empty axiom dependency list, including no propositional extensionality, quotient soundness, classical choice, or admitted proof.

This is a concrete contest proposal, not a claim of a mathematically largest definable natural number. Its comparison is independent of any unproved assignment of a named ordinal to the tree construction.

The submitted number and its certified margin. Let Ω_* be the explicit tree of Section 5 and H the hierarchy of Section 4. Submit

$$\boxed{N = H_{\Omega_*}(43).}$$

For $B(x) = 2^{x+44}$, the checked results include

$$\boxed{C_5 < N, \qquad B^{\,5}(C_5) \le N.}$$

The exponent 5 means five compositions of B, not an ordinary fifth power. This lower bound uses only a short path through the outer constructors of Ω_*.

Contents

1 The contest and the precise deliverable **3**
1.1 What the original rules require . . . 3
1.2 A frozen reference . . . 3
1.3 What a largest-number proposal can honestly mean . . . 3

2 The exact baseline **4**
2.1 Types, raw terms, and their sizes . . . 4
2.2 Why malformed terms matter . . . 4
2.3 Enumeration and the maximum . . . 5
2.4 Small checks and search-space size . . . 5

3 A constructive oracle diagonal **6**
3.1 An explicit language extension . . . 6
3.2 The reflection operator . . . 7
3.3 Escaping every fixed preceding program . . . 7

4 Interpreting well-founded trees **8**
4.1 A strictly positive type . . . 8
4.2 The numerical hierarchy . . . 8

5 The explicit tree construction **9**
5.1 Finite iteration and tree-indexed iteration . . . 9
5.2 The initial arithmetic-shaped operations . . . 10
5.3 Single and common closure recipes . . . 10
5.4 The indexed seed transformation . . . 11
5.5 Amplification at every finite functional type . . . 12
5.6 Promotion and the final closed tree . . . 13

6 Why the tree amplification gives numerical growth **14**

7 The comparison and a fivefold exponential margin **15**
7.1 The short strict-comparison proof . . . 15
7.2 A finite certificate inside the enormous tree . . . 15

8 Constructivity and the scope of the claims **17**
8.1 Why this is a constructive natural number . . . 17
8.2 Trees are not automatically canonical ordinals . . . 17
8.3 Exactly what the diagonal theorem says . . . 18
8.4 What is checked, and what remains a trust boundary . . . 18

9 Reproducible verification **18**
9.1 A pinned, small dependency base . . . 18
9.2 Axiom and dependency checks . . . 19
9.3 Small executable checks . . . 19
9.4 Commands to reproduce . . . 20

10 Guide to the artifacts **20**

11 Directions for a subsequent contender **21**

1 The contest and the precise deliverable

1.1 What the original rules require

The repository asks for constructive natural numbers, a formal definition, and a formal proof of a strict improvement. Its timing limits are 15 seconds for checking the definition and one minute for checking the comparison proof. It disallows axioms and discourages defining the next entry by directly using the previous contender as a constant [1]. These are *checking* limits; the rules do not require computing or printing the resulting numeral within those limits.

We retain these substantive requirements. The changed proof assistant is Lean, pinned to version 4.19.0, and the submitted names are `Biggest.contender_6` and `Biggest.contender_5_lt_contender_6`. Only `Init`, the standard Lean initialization library, is imported from outside the contest modules. No Mathlib installation is needed.

Our adapted axiom prohibition is literal: the transitive axiom dependencies of the number and its comparison proof must be empty. Merely avoiding newly declared axioms while using `Classical.choice` would not meet this rule. The code also contains no `sorry`, `admit`, `partial`, `unsafe`, `noncomputable`, or `native_decide`. Lean's documented axiom audit checks the dependencies [3].

1.2 A frozen reference

The reference is

`https://github.com/codyroux/name-the-biggest-number`

at commit

`d56dce013e6264292533efdc5e1de71863c6fb86`.

The comparison uses `contender_5` in `Contender.v`, attributed there to Samuel Grütter [2]. The original source and its MIT license are included for inspection.

There are two distinct assurance boundaries. The Coq-to-Lean correspondence is a constructor-by-constructor audit of the executable equations. We do not claim a mechanically verified translation between the two proof assistants. Once the faithful Lean baseline is fixed, the maximum characterization, enumeration theorems, comparison, diagonal results, and tree growth results are all proved and checked inside Lean.

1.3 What a largest-number proposal can honestly mean

No unrestricted collection of definable natural numbers has a largest member: a definition of m immediately makes a definition of $m+1$ possible. Resource-bounded contests constrain description and checking, but do not turn one construction into a proof of global optimality.

The entry below is the strongest concrete construction developed and validated in this work. It improves the *method* of producing numbers: it diagonally escapes a preceding language, iterates that escape through structural trees, and amplifies tree construction at all finite functional types. Its final value does not mention C_5. That constant appears only in comparison proofs. The actual compiled dependency graph is also audited.

2 The exact baseline

2.1 Types, raw terms, and their sizes

The original language has simple types

$$A, B ::= \mathbb{N} \mid A \to B$$

and raw terms

$$t ::= \mathsf{var}(i) \mid \lambda_{A,B} t \mid \mathsf{app}(t, u) \mid \mathsf{zero} \mid \mathsf{succ} \mid \mathsf{rec}_A.$$

A lambda records its domain and expected codomain. Natural recursion is available at every simple type:

$$\mathsf{rec}_A : A \to (\mathbb{N} \to A \to A) \to \mathbb{N} \to A.$$

This is a System-T-like language with a total, error-handling interpretation of *all raw terms*. The maximum is not restricted to well-typed terms.

Type and natural-index depths are

$$d_{\mathsf{Ty}}(\mathbb{N}) = 1, \quad d_{\mathsf{Ty}}(A \to B) = 1 + \max(d_{\mathsf{Ty}}(A), d_{\mathsf{Ty}}(B)), \quad d_{\mathbb{N}}(i) = i + 1.$$

The source term depths are exactly

$$\begin{aligned} d(\mathsf{var}(i)) &= i + 2, \\ d(\lambda_{A,B} t) &= 1 + \max(d_{\mathsf{Ty}}(A), d_{\mathsf{Ty}}(B), d(t)), \\ d(\mathsf{app}(t, u)) &= 1 + \max(d(t), d(u)), \\ d(\mathsf{zero}) &= d(\mathsf{succ}) = 1, \\ d(\mathsf{rec}_A) &= 1 + d_{\mathsf{Ty}}(A). \end{aligned}$$

A depth bound therefore bounds variable indices, annotations, and syntax, leaving only finitely many raw terms.

2.2 Why malformed terms matter

A value is a dependent pair (A, v), where

$$[\![\mathbb{N}]\!] = \mathbb{N}, \qquad [\![A \to B]\!] = [\![A]\!] \to [\![B]\!], \qquad v \in [\![A]\!].$$

Each type has a default value

$$\mathrm{err}_{\mathbb{N}} = 0, \qquad \mathrm{err}_{A \to B}(x) = \mathrm{err}_B\,.$$

A failed cast to B returns this default.

The source's cast operation is retained. Natural-to-natural casts are identity; casts between a natural and a function type fail. An arrow-to-arrow cast checks equality of both component type codes. If both tests succeed, the argument is transported backwards and the result forwards using recursively defined paired casts. Otherwise the result is the default function. The Lean `castImpl` follows these equations rather than replacing them with well-typed elaboration.

Environments are extended at the front, but indices count from the opposite end. If e has length ℓ, then

$$\mathrm{lookup}(e, i) = \begin{cases} \text{none}, & \ell \le i, \\ e[\ell - 1 - i], & i < \ell. \end{cases}$$

Thus index zero denotes the outermost binder. Substituting ordinary nearest-binder de Bruijn indexing would change the number.

An unbound variable returns natural zero. A lambda evaluates its body in the extended environment and casts to its declared codomain. Applying a natural value returns natural zero; applying a function casts its argument to the required domain. At the top level, natural results contribute their values and function results contribute zero.

A revealing example is

$$\mathrm{eval}(\mathsf{app}(\mathsf{succ}, \mathsf{succ})) = 1.$$

The argument is a function, so its cast to $\mathbb{N}$ gives zero; the outer successor returns one. An interpreter that rejects this term or immediately returns zero is not faithful to the source.

2.3 Enumeration and the maximum

Let $\mathcal{T}_n$ be `termsUpTo n`, with source order and repetitions retained. Lean proves

$$t \in \mathcal{T}_n \quad \Longleftrightarrow \quad d(t) \le n.$$

Lean theorem: `termsUpTo_correct`

Write $\max_0$ for finite maximum with value zero on an empty list, and set

$$C(n) = \max\nolimits_0 \{\mathrm{eval}(t) : t \in \mathcal{T}_n\}.$$

The original code selects a winning term with `maxBy`, starting from zero, then evaluates it. Replacement occurs only at a *strictly* larger value. The port retains this implementation as `baseline`; Lean proves its equality to the value-only maximum:

$$\mathsf{baseline}(n) = C(n).$$

Lean theorem: `maxBy_value; baseline_eq_maximum`

The exact reference contestant is

$$C_5 = C(42).$$

The comparison does not substitute an approximate bound for C_5.

2.4 Small checks and search-space size

Let T_n, Q_n count the enumerated type codes and terms. Their recurrences are

$$T_0 = Q_0 = 0, \qquad T_{n+1} = 1 + T_n^2,$$

$$Q_{n+1} = n + T_n^2 Q_n + Q_n^2 + 2 + T_n.$$

Direct Lean evaluation gives:

Depth n	Types T_n	Terms Q_n	$C(n)$
0	0	0	0
1	1	2	0
2	2	10	1
3	5	146	2
4	26	24,976	3

The recurrence already yields $Q_5 = 640,684,384$. This is a count calculation, not a claim that we enumerated that list. It illustrates why evaluating the depth-42 maximum is irrelevant to checking its definition.

3 A constructive oracle diagonal

3.1 An explicit language extension

The new language contains the original variable, lambda, application, zero, successor, and recursor constructors, plus

$$\mathsf{quote}(t), \qquad \mathsf{input}, \qquad \mathsf{oracle}.$$

Its evaluator has parameters $f : \mathbb{N} \to \mathbb{N}$ and $n \in \mathbb{N}$, with

$$\begin{aligned} [\![\mathsf{input}]\!]_{f,n} &= (\mathbb{N}, n), \\ [\![\mathsf{oracle}]\!]_{f,n} &= (\mathbb{N} \to \mathbb{N}, f), \\ [\![\mathsf{quote}(t)]\!]_{f,n} &= [\![t]\!]_{\mathrm{old},\,[]}. \end{aligned}$$

Quotation uses the empty *old* environment even beneath new binders. It preserves the whole old typed value, including a function value; final numerical extraction uses the previous rule.

Quotation is an ordinary constructor, not an elaborator reflection API or an unrestricted semantic constant. Its old syntax payload contributes to depth:

$$d_E(\mathsf{quote}(t)) = d(t) + 1, \qquad d_E(\mathsf{input}) = d_E(\mathsf{oracle}) = 1.$$

The other depth clauses are unchanged. Let $\mathcal{E}_n$ be `exprsUpTo n`. At stage $n+1$, its first sublist is $\{\mathsf{quote}(t) : t \in \mathcal{T}_n\}$; the remaining constructors use depth-n types, indices, and subexpressions. Lean proves

$$e \in \mathcal{E}_n \quad \Longleftrightarrow \quad d_E(e) \le n.$$

Lean theorem: `exprsUpTo_correct`

Write $E_f(e; n)$ for final numerical evaluation. In particular,

$$E_f(\mathsf{quote}(t); n) = \mathrm{eval}(t).$$

This equation is definitional.

Lean theorem: `eval_quote`

3.2 The reflection operator

Definition 3.1. For total $f : \mathbb{N} \to \mathbb{N}$, define

$$M_f(n) = \max_0\{E_f(e; n) : e \in \mathcal{E}_n\},$$

$$\boxed{\mathcal{R}(f)(n) = 2^{\max(f(n),M_f(n))+n+1}.} \tag{1}$$

This operation is computable for a supplied computable f: its list is finite and the interpreter is structurally total. No halting decision or unproved stopping condition is used.

The explicit $f(n)$ term provides uniform pointwise growth even at tiny depths. The exponential provides a convenient margin. The essential diagonal improvement comes from the bounded semantic maximum.

Lemma 3.2. *For every f, n,*

$$f(n) < \mathcal{R}(f)(n), \qquad e \in \mathcal{E}_n \Longrightarrow E_f(e; n) < \mathcal{R}(f)(n),$$

and

$$2^{f(n)+n+1} \le \mathcal{R}(f)(n).$$

Proof. Let $m = \max(f(n), M_f(n))$. Both $f(n)$ and any listed expression's value are at most m. The elementary inequality $r < 2^r$, with $m \le m + n + 1$, gives $m < 2^{m+n+1}$. The last inequality follows from monotonicity of powers of two in their exponent. □

Lean theorem: `reflect_gt_oracle; reflect_gt_member; oracle_power_le_reflect`

Lemma 3.3 (Uniform baseline comparison)**.** *For every f and r,*

$$C(r) < \mathcal{R}(f)(r + 1), \qquad 2^{C(r)+r+2} \le \mathcal{R}(f)(r + 1).$$

Proof. Every term in $\mathcal{T}_r$ has a quotation in $\mathcal{E}_{r+1}$ with the same numerical value. Thus $C(r) \le M_f(r + 1)$. Apply the preceding lemma and the defining exponent. No winning term needs to be found or evaluated. □

Lean theorem: `baseline_le_oracle_max; baseline_lt_reflect`

Lean theorem: `baseline_power_le_reflect`

The final input 43 now has an exact explanation: the baseline depth is 42, and quotation costs one additional constructor.

3.3 Escaping every fixed preceding program

Theorem 3.4 (Exact eventual domination)**.** *Fix $f : \mathbb{N} \to \mathbb{N}$ and one expression e. For every $n \ge d_E(e)$,*

$$E_f(e; n) < \mathcal{R}(f)(n).$$

Proof. Enumeration completeness gives $e \in \mathcal{E}_n$; apply Lemma 3.2. □

Lean theorem: `reflect_dominates_program`

The input primitive matters. It supplies the varying n at fixed syntax depth. A naive proof using the unary numeral for n instead would not justify this theorem, because that numeral's depth grows with n. Here the theorem concerns a fixed unary program with a fixed oracle.

Corollary 3.5. *There are no expression e and threshold b such that*

$$E_f(e; n) = \mathcal{R}(f)(n) \quad \textit{for all } n \geq b.$$

Proof. Take $n = \max(b, d_E(e))$ and contradict Theorem 3.4. □

Lean theorem: `no_fixed_program_defines_reflect`

Lean theorem: `no_program_eventually_defines_reflect`

Each successor stage below therefore supplies a total function beyond every fixed expression using the preceding oracle. This is not a claim that the function escapes all Lean definitions; its Lean definition is supplied in the package.

4 Interpreting well-founded trees

4.1 A strictly positive type

Use the inductive type

$$\mathsf{Stage} ::= \mathsf{z} \mid \mathsf{s}(a) \mid \mathsf{l}(s), \qquad s : \mathbb{N} \to \mathsf{Stage}.$$

Its Lean declaration is

```
inductive Stage where
  | zero : Stage
  | succ : Stage -> Stage
  | lim : (Nat -> Stage) -> Stage
```

The recursive occurrence in the codomain of $\mathbb{N} \to \mathsf{Stage}$ is strictly positive. Lean supplies a structural recursor and checks recursive definitions through recursor applications [4].

A limit node contains a total child function. It is not a cyclic graph or permission for unrestricted recursion. Every branch function in the submitted closed tree is explicitly constructed by the definitions below.

4.2 The numerical hierarchy

Define $H : \mathsf{Stage} \to \mathbb{N} \to \mathbb{N}$ by

$$H_{\mathsf{z}}(n) = n + 1, \tag{2}$$

$$H_{\mathsf{s}(a)}(n) = \mathcal{R}(H_a)(n), \tag{3}$$

$$H_{\mathsf{l}(s)}(n) = 1 + \max_{0 \leq k \leq n} H_{s(k)}(n). \tag{4}$$

The implementation samples $n, n-1, \ldots, 0$ using `natsUpTo (n+1)`. The order does not affect the maximum.

At a limit, all sampled children use the same external input n. Taking the maximum protects every sampled branch without requiring the branch sequence to be increasing. In particular,

$$k \le n \Longrightarrow H_{s(k)}(n) < H_{\mathsf{l}(s)}(n).$$

Thus each fixed branch is eventually strictly below its limit.

These equations are structural recursion on Stage, combined with a finite maximum and total reflection. They compute a natural for the explicit closed stage and input. We do not claim that every abstract semantic element of $\mathbb{N} \to \mathsf{Stage}$ in every classical set-theoretic model is computable. The particular submission has explicit computable branches and a checked total definition.

Lemma 4.1 (Local growth)**.** *For every a, s, n,*

$$H_a(n) < H_{\mathsf{s}(a)}(n), \qquad H_{s(0)}(n) < H_{\mathsf{l}(s)}(n).$$

Proof. Use the oracle inequality of Lemma 3.2 for a successor. At a limit, zero belongs to the finite sample and one is added to its maximum. □

Lean theorem: `numeric_successor_growth; numeric_limit_zero_growth`

5 The explicit tree construction

We now specify the single tree that indexes the submission. Every operation in this section corresponds to a transparent definition in `Trees.lean`; the equations fix the actual tree, including its branch order and its successor constructors. The construction has three parts: an initial family of tree transformations, amplification through all finite functional types, and promotion in which an amplified tree supplies an iteration index. The hierarchy H from Section 4 then turns the resulting tree into a natural number.

Throughout, Stage is the inductive type with constructors z, $\mathsf{s} : \mathsf{Stage} \to \mathsf{Stage}$, and $\mathsf{l} : (\mathbb{N} \to \mathsf{Stage}) \to \mathsf{Stage}$. These are literal constructors. In particular, a displayed limit includes its specified sequence of children. The construction uses no equivalence relation identifying different presentations, and every recursion below has an explicit structural argument. We retain suggestive source names such as `omegaPow` and `veblenTree` while defining their meaning entirely through these equations.

5.1 Finite iteration and tree-indexed iteration

For an endofunction $f : X \to X$ on any type X, write

$$f^0(x) = x, \qquad f^{k+1}(x) = f(f^k(x)).$$

This is `treeIter`; despite its name, its definition is polymorphic and its iteration counter is an ordinary natural number. We will use it both on trees and on functions between trees. Thus the superscript always denotes composition iteration, with the underlying type determined by the supplied function.

A separate operation, `stageIter`, uses a tree as its index. For $f : \mathsf{Stage} \to \mathsf{Stage}$, define

$$\begin{aligned} I_f(\mathsf{z}, x) &= x, \\ I_f(\mathsf{s}(t), x) &= f(I_f(t, x)), \\ I_f(\mathsf{l}(s), x) &= \mathsf{l}\big(k \mapsto I_f(s(k), x)\big). \end{aligned} \tag{5}$$

The recursion is on the first tree argument t. The starting tree x remains a parameter, and each limit branch makes the recursive call at its corresponding child. No comparison between x and t is required. In particular, the later expression $I_f(t, t)$ is a legitimate application of an already defined structural iterator.

The finite trees and the first branching tree are

$$\widehat{0} = \mathsf{z}, \qquad \widehat{k+1} = \mathsf{s}(\widehat{k}), \qquad \omega_{\mathsf{Stage}} = \mathsf{l}(k \mapsto \widehat{k}).$$

They are `stageNat` and `stageOmega`. Keeping the two iteration operations distinct will matter when finite iteration is applied to a transformation of functions, and the result is subsequently used inside a tree-indexed iteration.

5.2 The initial arithmetic-shaped operations

Write $a \oplus b$ for `stageAdd a b` and $a \otimes b$ for `stageMul a b`. Their defining equations are

$$\begin{aligned} &a \oplus \mathsf{z} = a, && a \oplus \mathsf{s}(b) = \mathsf{s}(a \oplus b), && a \oplus \mathsf{l}(s) = \mathsf{l}(k \mapsto a \oplus s(k)), \\ &a \otimes \mathsf{z} = \mathsf{z}, && a \otimes \mathsf{s}(b) = (a \otimes b) \oplus a, && a \otimes \mathsf{l}(s) = \mathsf{l}(k \mapsto a \otimes s(k)). \end{aligned}$$

Both recurse on the right argument. The order in the multiplication successor clause is part of the specification: the accumulated result appears on the left of $\oplus$, with the original a on its right. No commutativity or algebraic simplification is being assumed.

Let $p : \mathsf{Stage} \to \mathsf{Stage}$ denote `omegaPow`. It is defined by

$$p(\mathsf{z}) = \mathsf{s}(\mathsf{z}), \qquad p(\mathsf{s}(a)) = p(a) \otimes \omega_{\mathsf{Stage}}, \qquad p(\mathsf{l}(s)) = \mathsf{l}(k \mapsto p(s(k))).$$

These equations provide a concrete initial transformation for subsequent construction. The names and notation record the motivation for the recipe; the submitted number requires only the stated operations on Stage. We do not replace any of these trees by an ordinal representative or invoke arithmetic laws about such representatives.

5.3 Single and common closure recipes

For a tree transformation f, first define the tree

$$O_f(x) = \mathsf{l}(k \mapsto f^k(x)).$$

This is `omegaClose f x`. Its zeroth child is exactly x. The word "closure" names this construction; it does not assert that applying f leaves the resulting tree unchanged.

Given any transformation $c : \mathsf{Stage} \to \mathsf{Stage}$, define its enumeration $E_c : \mathsf{Stage} \to \mathsf{Stage}$ by

$$\begin{aligned} E_c(\mathsf{z}) &= c(\mathsf{z}), \\ E_c(\mathsf{s}(a)) &= c\big(\mathsf{s}(E_c(a))\big), \\ E_c(\mathsf{l}(s)) &= \mathsf{l}(k \mapsto E_c(s(k))). \end{aligned}$$

This is `enumerateClosure`. The successor inside its middle clause is deliberate. The transformation called `derivative f` is

$$D(f) = E_{O_f}.$$

These definitions make sense for every supplied total f and c, without hypotheses of monotonicity, continuity, or fixed-point behavior. For example, the first two enumerated values of the derivative satisfy

$$D(f)(\mathsf{z}) = O_f(\mathsf{z}), \qquad D(f)(\mathsf{s}(\mathsf{z})) = O_f\big(\mathsf{s}(O_f(\mathsf{z}))\big).$$

The second value therefore starts a fresh iteration sequence from the successor of the first constructed tree. It is not obtained by merely selecting a later child of the original sequence. This distinction fixes where the successor constructors enter the seed recipe and prevents an informal fixed-point interpretation from erasing part of its syntax.

We also require a specified recipe for a sequence $\mathbf{f} = (f_0, f_1, \ldots)$ of tree transformations. Its finite rounds and accumulated orbit are

$$r_0^{\mathbf{f}}(x) = x, \qquad r_{m+1}^{\mathbf{f}}(x) = f_m(r_m^{\mathbf{f}}(x)),$$
$$u_0^{\mathbf{f},x} = x, \qquad u_{m+1}^{\mathbf{f},x} = r_{m+1}^{\mathbf{f}}(u_m^{\mathbf{f},x}).$$

These are `closureRound` and `closureOrbit`. Round m applies f_0 first and f_{m-1} last. The orbit repeatedly starts the next, longer round from the result already obtained. For example, its first two noninitial values are $u_1 = f_0(x)$ and $u_2 = f_1(f_0(f_0(x)))$. This example fixes both the order of composition and the repeated application of earlier functions.

Finally set

$$J_{\mathbf{f}}(x) = \mathsf{l}(m \mapsto u_m^{\mathbf{f},x}), \qquad D_*(\mathbf{f}) = E_{J_{\mathbf{f}}}.$$

The corresponding names are `commonClose` and `commonDerivative`. They are explicit branching-tree operations; their use does not require a separate theorem about common fixed points.

5.4 The indexed seed transformation

For a base transformation $b : \mathsf{Stage} \to \mathsf{Stage}$, define a family $V_b(a) : \mathsf{Stage} \to \mathsf{Stage}$, structurally indexed by $a \in \mathsf{Stage}$, through

$$V_b(\mathsf{z}) = b,$$
$$V_b(\mathsf{s}(a)) = D(V_b(a)),$$
$$V_b(\mathsf{l}(s)) = D_*(k \mapsto V_b(s(k))).$$

This family is `veblenTree base`. At a successor index the previous transformation is passed to the single-transformation recipe; at a limit index the whole sequence of recursively obtained transformations is passed to the common recipe. The definitions specify actual functions, so no choice of a canonical index or a preferred branch sequence occurs.

Define the transformation

$$K_b = D\big(a \mapsto V_b(a)(\mathsf{z})\big).$$

This is `criticalLift base`. The tree used as the initial argument later in the construction is

$$\gamma = K_p(\mathsf{z}), \tag{6}$$

which is exactly `gammaTree`. The displayed γ is a name for this tree. It carries no asserted identification with a named ordinal or a named growth rate. All subsequent operations can therefore be checked using this definition alone.

5.5 Amplification at every finite functional type

The next part builds transformations uniformly through the finite types

$$X_0 = \mathsf{Stage}, \qquad X_{r+1} = X_r \to X_r.$$

These are the types `TreeLevel r`. A value at level two transforms tree endofunctions; a value at level three transforms those transformations, and the recursion continues through every natural level.

Define a sequence-combining operation $L_r : (\mathbb{N} \to X_r) \to X_r$ and a successor operation $S_r : X_r \to X_r$ by

$$\begin{aligned} L_0(s) &= \mathsf{I}(s), & S_0(x) &= \mathsf{s}(x), \\ L_{r+1}(s)(x) &= L_r(k \mapsto s(k)(x)), & S_{r+1}(g)(x) &= S_r(g(x)). \end{aligned}$$

They are `levelLimit` and `levelSucc`. Above level zero they act pointwise. Thus the apparent limit of functions is constructed by lambda abstraction and the lower-level operation. No limit in a topology, existence theorem, or extensional equality proof is needed to define it.

For each r, the amplifier $A_r \in X_{r+2}$ is

$$A_r(f)(x) = S_r\big(L_r(k \mapsto f^k(x))\big), \qquad f \in X_{r+1}, \quad x \in X_r. \tag{7}$$

This is `omegaAmplifier r`. The finite iteration is well typed because f is an endofunction on X_r. The successor is applied after the sequence-combining operation at the same level. Retaining this successor is material to the growth proofs; deleting it would define a different candidate. The construction does not assume that successive type levels produce numerically increasing values.

The first higher functional level makes the typing especially visible. For $F : X_1 \to X_1$, a tree transformation $f \in X_1$, and a tree a, the pointwise equations give

$$A_1(F)(f)(a) = \mathsf{s}\big(\mathsf{I}(k \mapsto (F^k(f))(a))\big).$$

Here the finite iteration acts on transformations before the resulting transformation is applied to a. In contrast, $A_0(f)(a)$ iterates f directly on trees. This explains why the varying type level adds structure beyond increasing the natural iteration counter in a single tree transformation. Both expressions nevertheless reduce to values of the same raw tree type once their arguments have been supplied.

To turn these higher-type values back into trees, fix $f : \mathsf{Stage} \to \mathsf{Stage}$ and $a \in \mathsf{Stage}$. Define seeds

$$\sigma_0^{f,a} = a, \qquad \sigma_1^{f,a} = f, \qquad \sigma_{r+2}^{f,a} = A_r,$$

and descent maps $C_r^{f,a} : X_r \to \mathsf{Stage}$ by

$$C_0^{f,a}(x) = x, \qquad C_{r+1}^{f,a}(g) = C_r^{f,a}\big(g(\sigma_r^{f,a})\big).$$

They are `levelSeed` and `descendLevel`. Each descent step applies one function to a seed of exactly the required type, then decreases the level. It does not inspect a function's implementation or search for a suitable argument.

The resulting tree transformation is

$$T_f(a) = \mathsf{s}\Big(\mathsf{I}\big(r \mapsto C_{r+2}^{f,a}(A_r)\big)\Big). \tag{8}$$

This is `higherTower f a`. Each branch has a finite type level, although the branch function admits every level. At branch zero, descent unfolds to

$$C_2^{f,a}(A_0) = A_0(f)(a) = \mathsf{s}\big(\mathsf{I}(k \mapsto f^k(a))\big).$$

This identity will supply a short, concrete growth argument without requiring a comparison between different higher-type branches.

5.6 Promotion and the final closed tree

The promotion operation P transforms a tree endofunction into a new tree endofunction:

$$P(f)(a) = \mathsf{s}\big(I_{T_f}(T_f(a), a)\big). \tag{9}$$

It is `promoteTreeFunction`. Both roles of T_f are explicit: $T_f(a)$ supplies the tree index, while T_f itself is the transformation being iterated. This reuse creates no recursive equation for an unknown object. First T_f is constructed, and then the previously defined iterator is applied to it and to a constructed tree.

Set

$$f_0(a) = \mathsf{s}(K_p(a)), \tag{10}$$

$$G(a) = \mathsf{s}\Big(\mathsf{I}\big(k \mapsto (P^{k+1}(f_0))(a)\big)\Big). \tag{11}$$

The source names are `startingTreeFunction` and `promotedTreeFunction`. Here P^{k+1} is ordinary finite iteration on the type $X_1 = \mathsf{Stage} \to \mathsf{Stage}$. The zeroth branch is $P(f_0)(a)$, not $f_0(a)$: the exponent $k+1$ is essential to the stated definition and to the uniform growth proof that follows. The functions in these branches need not be ordered against one another.

With this fixed G, the remaining choices are

$$\boxed{s = G(\gamma), \qquad \Omega_* = \mathsf{s}\big(I_G(s, s)\big), \qquad N = H_{\Omega_*}(43).} \tag{12}$$

The same constructed tree s supplies both the index and starting argument of the final structural iteration. In the source, the last tree definition is exactly

```
def finalStage : Stage :=
  let seed := promotedTreeFunction gammaTree
  .succ (stageIter promotedTreeFunction seed seed)
```

The natural-number declaration is `contender_6 := hierarchy finalStage 43`.

The input 43 connects this construction to the reference comparison: quoting any old term of depth at most 42 costs one additional depth level. Every reflection at input 43 therefore includes that entire reference family. The tree recipe itself contains no reference to the previous contender. Its growth mechanism remains the same at other numerical inputs; the following section proves the relevant strict inequalities from local successor and limit behavior of H.

This completes the finite specification. Natural recursion handles finite rounds and type levels, and structural tree recursion handles the tree arguments. Infinitely many possible branches are represented by total functions with finite definitions. The claim is that these precise operations

produce the stated checked natural number; assigning an ordinal meaning or an externally named asymptotic scale is a separate question and supplies no premise of the comparison proof.

There are also two distinct uses of successor throughout the construction. The constructor s and its pointwise extensions S_r build trees or functions returning trees. Only after the final tree is supplied to H does a tree successor request the numerical reflection operation $\mathcal{R}$. The tree transformations do not evaluate the numerical hierarchy while choosing their indices. This separation keeps the complete recipe typed and total independently of the size of the natural number obtained from it.

6 Why the tree amplification gives numerical growth

The tree formulas are elaborate, so their intended numerical effect needs proof rather than an appeal to their resemblance to ordinal notation. A useful abstraction separates this question from the details of reflection.

Let $v : \mathsf{Stage} \to \mathbb{N}$ satisfy just

$$v(a) < v(\mathsf{s}(a)), \qquad v(s(0)) < v(\mathsf{l}(s)). \tag{13}$$

For each fixed n, the interpretation $v(a) = H_a(n)$ satisfies these conditions by Lemma 4.1. No ordering of arbitrary trees, monotonicity of the oracle, or monotonicity of a limit's branches is assumed.

Theorem 6.1 (Checked amplification properties)**.** *For every v satisfying (13), every function $f : \mathsf{Stage} \to \mathsf{Stage}$, and every $a \in \mathsf{Stage}$,*

$$v(a) < v(T_f(a)), \qquad v(a) < v(P(f)(a)), \qquad v(a) < v(G(a)).$$

Moreover, $v(\gamma) < v(\Omega_*)$, where γ is the tree called `gammaTree`*.*

Proof. The zeroth branch in $T_f(a)$, after the descent operation is unfolded, is

$$\mathsf{s}\big(\mathsf{l}(k \mapsto f^k(a))\big).$$

The zeroth child of its limit is $f^0(a) = a$. Two applications each of successor and limit growth establish the first claim. This argument does not require f itself to increase v.

If $v(x) \le v(h(x))$ for every x, structural induction on t shows

$$v(a) \le v(I_h(t, a)).$$

The zero case is equality. A successor applies extensivity of h after the induction hypothesis. At a limit, use its zeroth branch and the second hypothesis in (13).

Apply this observation to $h = T_f$, which is extensive by the first claim. The outer successor in $P(f)(a) = \mathsf{s}(I_{T_f}(T_f(a), a))$ makes the inequality strict. For G, its zeroth limit branch is $P(f_0)(a)$; apply the second claim, followed by limit and successor growth.

Finally let $s = G(\gamma)$. The third claim gives $v(\gamma) < v(s)$ and makes G extensive. Consequently

$$v(\gamma) < v(s) \le v(I_G(s, s)) < v(\Omega_*).$$

□

Lean theorem: `higherTower_increases; stageIter_extensive`

Lean theorem: `promoteTreeFunction_increases; promotedTreeFunction_increases`

Lean theorem: `finalStage_exceeds_gammaTree`

Specializing this theorem to $v(a) = H_a(n)$ gives, for every input n,

$$H_a(n) < H_{T_f(a)}(n), \quad H_a(n) < H_{P(f)(a)}(n), \quad H_a(n) < H_{G(a)}(n), \quad H_\gamma(n) < H_{\Omega_*}(n).$$

These specializations are separate checked theorems in `NumericTreeGrowth.lean`. They establish actual numerical growth. They do not identify a standard fast-growing-hierarchy index, prove that $P(f)(a)$ dominates $f(a)$, or compare each adjacent finite functional level with the next.

7 The comparison and a fivefold exponential margin

7.1 The short strict-comparison proof

The final tree has an outer successor. Thus $N = \mathcal{R}(H_b)(43)$ for its explicitly defined predecessor b, and Lemma 3.3, with $r = 42$, immediately yields $C_5 < N$. The complete comparison theorem is:

```
theorem contender_5_lt_contender_6 :
    contender_5 < contender_6 := by
  unfold contender_6 finalStage
  exact baseline_lt_reflect _ 42
```

This proof is short because a uniform comparison with every oracle has already been proved. Checking the supporting definitions and lemmas is included in the reported budgets; the timing does not merely measure these three final lines.

7.2 A finite certificate inside the enormous tree

For a baseline depth r, put

$$B_r(x) = 2^{x+r+2}, \qquad B_r^0(x) = x, \qquad B_r^{k+1}(x) = B_r(B_r^k(x)).$$

The relation $\mathsf{MarginPath}(k, a)$ records a finite route through a containing k successor edges, with only zeroth-child moves at limit nodes. It has the rules

$$\mathsf{MarginPath}(0, a), \qquad \frac{\mathsf{MarginPath}(k, a)}{\mathsf{MarginPath}(k+1, \mathsf{s}(a))}, \qquad \frac{\mathsf{MarginPath}(k, s(0))}{\mathsf{MarginPath}(k, \mathsf{l}(s))}.$$

The endpoint of the route may be any tree. We need neither unfold the whole tree nor traverse every branch.

Lemma 7.1 (Path bound)**.** *If $k \neq 0$ and $\mathsf{MarginPath}(k, a)$, then*

$$B_r^k(C(r)) \leq H_a(r+1).$$

Proof. Induct on the path certificate. A path of length zero is excluded by the hypothesis. At the innermost successor, use the uniform bound

$$B_r(C(r)) \le \mathcal{R}(H_b)(r+1),$$

which holds for every predecessor b, regardless of its value. At each remaining successor, combine the induction hypothesis, monotonicity of the ordinary function B_r, and

$$B_r(H_b(r+1)) \le H_{\mathsf{s}(b)}(r+1).$$

At each zeroth limit move, the corresponding child's value is bounded by the finite maximum defining its parent. □

Lean theorem: `margin_path_bound`

The following certificate checks by unfolding only the required outer constructors:

```
theorem finalStage_five_successors :
    MarginPath 5 finalStage := by
  apply MarginPath.next
  apply MarginPath.next
  apply MarginPath.limit0
  apply MarginPath.next
  apply MarginPath.next
  apply MarginPath.limit0
  apply MarginPath.next
  exact MarginPath.base _
```

The moves have the following provenance:

Successor counted	Constructor exposed along the chosen route
1	The outer successor of Ω_*
2	The outer successor of G, exposed by the tree iterator
3	The outer successor of $P(f_0)$, selected at branch zero of G
4	The outer successor of T_{f_0}, exposed by its iterator
5	The successor in the level-zero amplifier, after descent

Limit moves between these successors use only branch zero, which is always among the sampled branches. This route certifies only a small part of the definition's growth.

Theorem 7.2 (Certified numerical margin)**.** *Define* $x_0 = C_5$ *and* $x_{j+1} = 2^{x_j+44}$. *Then*

$$\boxed{x_5 \le N.}$$

Proof. Apply Lemma 7.1 to $r = 42$, $k = 5$, and the displayed certificate. □

```
theorem contender_6_fivefold_margin :
    marginTower 42 5 (baseline 42) <= contender_6 :=
  margin_path_bound 42 finalStage_five_successors
    (Nat.noConfusion)
```

Here `baseline 42` is exactly the defining body of `contender_5`. The formulation avoids needlessly asking the elaborator to normalize the huge closed number.

The fivefold bound is a rigorous lower bound, not an estimate of N, an upper bound, or a claim that five exponentials capture its scale. It is much stronger than the strict comparison required by the contest.

8 Constructivity and the scope of the claims

8.1 Why this is a constructive natural number

Every operation in the submitted closed definition is built from inductive constructors, total functions, structural recursors, finite lists, finite maxima, and natural-number arithmetic. Although l stores a function with infinitely many possible arguments, it is represented by a finite piece of executable code. At a numerical limit stage, only finitely many arguments are requested.

A subtle point is that reflection may call its oracle at an input larger than the current n. This does not threaten termination: the recursively available object is the entire function $H_a : \mathbb{N} \to \mathbb{N}$ at a structurally smaller stage. The recursion is on the stage, not on a claim that every oracle call decreases its natural input. The Lean kernel accepts the resulting recursor-based definition.

The submitted definition and proofs depend on no termination oracle, choice principle, or semantic assertion that arbitrary programs halt. In particular, the construction does not use Busy Beaver values. Finite syntactic enumeration of a language with a total interpreter is sufficient. The distinction between finite descriptions, enormous values, and uncomputable growth is central to large-number contests; Aaronson's exposition gives broader context [7].

Computable in principle does not mean feasible to evaluate. Neither C_5 nor N was expanded to a numeral, and none of the verification commands attempts that calculation. The task is to check a finite definition and finite proofs about its value.

8.2 Trees are not automatically canonical ordinals

The constructors of `Stage` resemble zero, successor, and countable limits. The seed constructions deliberately resemble familiar ordinal operations. We nevertheless use raw trees throughout, with no quotient by ordinal equality, no certified normal-form predicate, and no proof that a branch sequence is cofinal in a particular ordinal.

This distinction matters numerically. For example,

$$H_{\mathsf{z}}(n) = n + 1, \qquad H_{\mathsf{l}(k \mapsto \mathsf{z})}(n) = n + 2.$$

A conventional ordinal supremum would identify the second tree's putative ordinal value with zero, but our numerical semantics distinguishes the two presentations. Theorems about canonical ordinal notation cannot simply be transferred to these trees.

Accordingly, `gammaTree` is a descriptive source name; this article does not prove that it denotes Γ_0. We claim no exact Bachmann–Howard or other named ordinal calibration for Ω_*, and no associated proof-theoretic lower bound. Formal relationships between different constructive notions of ordinals require real additional hypotheses and proofs; see Kraus, Nordvall Forsberg, and Xu [6]. Our comparison and amplification theorems avoid that unproved step.

8.3 Exactly what the diagonal theorem says

Theorem 3.4 fixes one expression and one total oracle. For every sufficiently large input, reflection exceeds that expression evaluated with that input and oracle. The finite depth of the fixed expression supplies the explicit threshold. The theorem neither quantifies over every Lean program nor asserts that all oracle programs are increasing.

In particular, pointwise $f \leq g$ does not by itself justify $\mathcal{R}(f) \leq \mathcal{R}(g)$: programs may inspect or transform oracle outputs in ways that fail to preserve order. None of our proofs requires this inference. We use the explicit guard $f(n)$, quotation membership, and branches actually sampled by a finite maximum.

Similarly, the tree proofs establish the specific inequalities in Theorem 6.1. Their strength is that they hold for every supplied f without auxiliary monotonicity assumptions. They should not be read as a classification of the entire hierarchy by growth rate.

8.4 What is checked, and what remains a trust boundary

The Lean kernel checks the internal theorems. The source-to-source correspondence with the original Coq baseline is a detailed manual audit supported by exact source preservation and small execution checks. A fully formal translation between Coq's and Lean's foundations is not part of this deliverable.

An empty axiom list means the checked declaration has no axiom dependencies in Lean's environment. It still presupposes the ordinary correctness of the kernel and the environment used to execute it. The package records the exact compiler version, source hashes, commands, and audit output so that the result can be checked independently on a normal installation.

9 Reproducible verification

9.1 A pinned, small dependency base

The contest modules use Lean 4.19.0 and import only `Init` from its standard distribution [5]. All other imports refer to the included source modules. The external dependency-inspection tool additionally imports Lean's official metaprogramming API to read compiled declarations. That tool is part of the verification harness, not part of the submitted number's definition.

The verifier compiles all project modules afresh in a temporary directory, without using prebuilt project object files. It records process wall time for every module. Its definition budget conservatively includes the baseline and shared interpreter, the oracle language, all tree definitions, and the hierarchy. Its proof budget includes the comparison's supporting proofs, the auxiliary theorems, the small checks, the generated axiom audit, and the compiled dependency inspection.

Verification item	Recorded result
Definition modules, fresh aggregate	1.404626 seconds
Proofs, examples, and both audits	2.741057 seconds
Inventoried declarations with no axioms	144
Constants reachable from the submitted value	444
Previous contender reachable	No
Baseline maximum reachable	No
Small executable check groups	9 of 9 matched

The run completed on 13 October 2026 using the official Lean 4.19.0 release (commit `6caaee842e94`), Linux x86_64 with glibc 2.39, and Python 3.12.14. The recorded compiler executable SHA-256 is

`92c3d35b5bfaa5e0fea413a775d504cf46cd95e1345df61c2274f76779e7e023`.

The complete JSON result includes each module's wall time and output, all source hashes, the 144 axiom-audit results, and the compiled project-dependency closure.

These are observed measurements on the recorded environment, not a promise that any machine, under any load, achieves the same times. They give substantial room within the adopted 15-second definition and 60-second proof limits.

9.2 Axiom and dependency checks

The harness first rejects forbidden source constructs such as `sorry` and `unsafe`, ignoring comments and string literals. This is only an early diagnostic. Its decisive logical check generates `#print axioms` commands for the public definitions, inductive declarations, and theorems and verifies that each reported dependency list is empty. Critical results are explicitly required, so a missing comparison or margin theorem cannot accidentally pass as a vacuous audit.

The definition of the new number must also be independent of the previous contestant constant. The harness traverses the actual compiled constants reachable from `Biggest.contender_6`, including types and declaration bodies. It confirms that neither `Biggest.contender_5` nor `Biggest.baseline` is reachable. The recorded closure also omits `Biggest.largestOfDepth` and `Biggest.maxBy`. The shared raw syntax, interpreter, and uniform enumerators are intentionally reachable: they implement the language being extended, not a use of the old winning number.

The distinction is semantic. Merely importing a source module that also declares `contender_5` does not make that constant a dependency of the new value. Conversely, hiding the constant behind a helper definition would still be found by the compiled closure traversal.

9.3 Small executable checks

The verified sample records include the type and term counts and baseline values in Section 2, expression counts $0, 4, 28, 914$ at depths zero through three, raw-interpreter samples, quotation preservation, oracle/input behavior, and small reflection values. Their purpose is to catch porting and wiring errors that readable examples expose well. They do not replace the general enumeration and comparison proofs.

No high-depth term list or large tree value is evaluated. The checker verifies the finite syntax of the submission and its proof terms; this is precisely the resource being measured.

9.4 Commands to reproduce

Install the standard Lean 4.19.0 distribution and use Python 3 with its standard library. From the archive root, run:

```
python3 verification/verify.py \
  --lean /path/to/lean-4.19.0/bin/lean \
  --source-dir lean \
  --output verification/reproduced-results.json
```

The `lean-toolchain` file also pins the release for an ordinary Elan installation. The supplied verifier builds dependencies in their correct order and does not require Lake or Mathlib.

To rebuild the article, use a TeX installation containing XeLaTeX, Latin Modern and DejaVu fonts, and the packages named in the preamble:

```
cd article
latexmk -xelatex -interaction=nonstopmode \
  -halt-on-error reflection_tree_contender.tex
```

The checked source distribution contains the main article, its three included `.tex` files, the compiled PDF, and the independent verification evidence.

The execution container needed one narrowly scoped compatibility helper because its PID namespace and mounted procfs did not agree: Lean's lookup of its own executable used a pathname unavailable in that namespace. The included C source redirects exactly that own-executable lookup to `/proc/self/exe`. The official compiler and shared libraries were unchanged. This affects executable discovery, not logical constants or kernel checking. A normal installation does not need it; full details and binary hashes are recorded in the runtime notes.

10 Guide to the artifacts

File or directory	Role
`lean/Baseline.lean`	Exact raw interpreter, enumeration, maximum, and the reference constant.
`lean/Enumeration.lean`	Soundness and completeness of the original finite enumerators.
`lean/Oracle.lean`	Quotation, input, oracle syntax, and the reflection operator.
`lean/OracleFacts.lean`	Uniform oracle growth and baseline comparison.
`lean/Diagonal.lean`	Exact expression enumeration and eventual diagonal domination.
`lean/Trees.lean`	All tree operations and the explicit final stage.
`lean/Hierarchy.lean`	Numerical tree interpretation and `contender_6`.
`lean/TreeGrowth.lean`	Generic structural amplification proofs.
`lean/TreeMargin.lean`	Finite path certificate and the fivefold exponential bound.
`lean/NumericTreeGrowth.lean`	Growth theorems instantiated for the actual hierarchy.

File or directory	Role
`lean/Comparison.lean`	The strict comparison with `contender_5`.
`lean/Checks.lean`	Small executable regression examples.
`lean/Main.lean`	Convenient import point and principal theorem audits.
`verification/`	Fresh-build script, measured results, logs, and runtime notes.
`reference/`	Frozen upstream files, their license, source hashes, and correspondence notes.
`article/`	Full article source and compiled PDF.

11 Directions for a subsequent contender

The present construction is frozen after proof and review. Several substantive extensions could support a later contest entry.

One route is to develop an explicit ordinal-notation invariant, prove the required branch properties, and connect the resulting numerical hierarchy to a standard growth classification. That would make named ordinal comparisons justified rather than suggestive. A separate route is to index the functional-type construction by trees, using dependent products at limit types; this would require a compatible descent and numerical majorization theory.

Another improvement is to extend the total object language while preserving a proved interpreter and finite, complete enumeration. The reflection proof is modular: a language embedding that respects values and has a controlled depth cost immediately supplies a new comparison. A formal Coq-to-Lean simulation would strengthen the current source-correspondence boundary independently of making the number larger.

For the existing entry, richer finite path certificates could prove stronger lower bounds without changing its definition. They would refine what is known about the same N, rather than estimate its full value. Efficient duplicate elimination in enumeration could also be justified by proving preservation of the semantic maximum, although it is unnecessary for definition checking.

There is no canonical last enlargement. The contribution here is a precise closed term with an explicit improvement mechanism, a strict comparison with the repository's actual baseline, and a reproducible record of the proofs and resource measurements.

References

[1] Cody Roux and contributors. *Name the biggest number: contest rules.* Repository README, frozen at commit `d56dce013e6264292533efdc5e1de71863c6fb86`. https://github.com/codyroux/name-the-biggest-number.

[2] Cody Roux and contributors; Contender 5 by Samuel Grütter. *Contender.v.* Same frozen commit; an exact copy is included in this archive. https://github.com/codyroux/name-the-biggest-number/blob/d56dce013e6264292533efdc5e1de71863c6fb86/Contender.v.

[3] Lean development team. *The Lean Language Reference: Axioms.* https://lean-lang.org/doc/reference/latest/Axioms/. Accessed 12 October 2026; verification itself is pinned to 4.19.0.

[4] Lean development team. *The Lean Language Reference: Inductive Types.* https://lean-lang.org/doc/reference/latest/The-Type-System/Inductive-Types/. Accessed 12 October 2026.

[5] Lean development team. *Lean 4.19.0 release.* https://github.com/leanprover/lean4/releases/tag/v4.19.0.

[6] Nicolai Kraus, Fredrik Nordvall Forsberg, and Chuangjie Xu. *Type-Theoretic Approaches to Ordinals.* 2022. https://arxiv.org/abs/2208.03844.

[7] Scott Aaronson. *Who Can Name the Bigger Number?* 1999. https://www.scottaaronson.com/writings/bignumbers.html.